

- Set operators combine the results of two or more SELECT queries.
- Oracle provides four main set operators to work with result sets.
 - UNION
 - UNION ALL
 - INTERSECT
 - MINUS

syntax

- SELECT column1, column2, ...
- FROM table1
- SET_OPERATOR
- SELECT column1, column2, ...
- FROM table2;

- SELECT emp_id, emp_name, department, salary
- FROM hr_employees
- UNION
- SELECT emp_id, emp_name, department, salary
- FROM it_employees
- UNION
- SELECT emp_id, emp_name, department, salary
- FROM sales_employees
- ORDER BY emp_id;

- SELECT emp_id, emp_name, department, salary, 'HR' as source
- FROM hr_employees
- UNION ALL
- SELECT emp_id, emp_name, department, salary, 'IT' as source
- FROM it_employees
- UNION ALL
- SELECT emp_id, emp_name, department, salary, 'Sales' as source
- FROM sales_employees
- ORDER BY emp_id;

INTERSECT - Employees who exist in multiple departments

- -- Employees in both HR and IT
- SELECT emp_id, emp_name
- FROM hr_employees
- INTERSECT
- SELECT emp_id, emp_name
- FROM it_employees;

- -- HR employees not in all_employees
- SELECT emp_id, emp_name, department
- FROM hr_employees
- MINUS
- SELECT emp_id, emp_name, department
- FROM all_employees;

What is an Index ?

- An **index** is a database object that improves the **speed of data retrieval** on a table.
- It works like the index of a book — instead of scanning every row (full table scan), Oracle uses the index to locate rows faster.

- `select emp_id,emp_name,department from employees; --
cjnvs28u7sj9v`
- `select * from v$sql;`
- `select * from v$sql_plan;`
- `select * from v$sql where sql_text like('select
emp_id,emp_name,department from employees%');`
- `select * from v$sql_plan where sql_id='cjnvs28u7sj9v';`

Types of Index

- B-Tree Index
- Bitmap Index
- Composite Index
- Unique Index
- Function Based Index
- Index Organized table

B-Tree Index

- Balanced Tree Index
- Its an ordered list of values that are divided in to range associated with rowid.
- It is recommended to create the index on high cardinality columns.
- **High cardinality** → many unique values

Bitmap Index

- It stores the data in the form of bitmap array, stores the index key column information along with your rowid.
- Its is recommended to create on low cardinality columns.

Composite Index

- Created on multiple columns in a table.
- It is also called as concatenated index.

Unique index

- Doesn't allow duplicate values and allow null values

Function Based Index

- If an index created using function or expression the it is said to be function based index.
- It can be either B-Tree or Bitmap.

Index organized Table

- Organizing the data at the time of inserting.
- You must have primary key on the table.

View

- View is a logical/virtual table that doesn't physically exist in the database and is stored in the Oracle data dictionary.
- View stores SQL Statements/Queries in the database.
- It will not hold any data of its own; instead, it retrieves/fetches the data from the base table/view whenever it is required.
- A view looks and acts like a table.
- No data is stored in a view.

- Business Intelligence
- Need of Business Intelligence
- Terms used in BI
- Components of BI

- Business Intelligence is the process of taking raw data from your business and turning it into meaningful and useful information that enables better, data-driven decision-making.

- BI is a technology-driven process for analyzing data and presenting actionable information to help corporate executives, business managers, and other end users make informed business decisions.
- It encompasses a wide range of tools, applications, and methodologies that allow organizations to collect data from internal systems and external sources, prepare it for analysis, develop and run queries against that data, and create reports, dashboards, and data visualizations.

- Think of BI as the dashboard of a car. A car's dashboard takes raw data from the engine (RPM, speed, fuel level, engine temperature) and presents it to the driver in an easy-to-understand format.
- The driver doesn't need to know the intricate mechanics; they just need to see the speedometer to know if they're speeding or the fuel gauge to know when to refill.
- Similarly, BI takes complex business data and presents it in a way that managers can quickly understand what's happening and what to do next.

The Need for Business Intelligence

- **Informed Decision-Making:** This is the core need. BI replaces intuition with facts. Instead of guessing which marketing campaign worked best, BI can show you precisely which one generated the highest ROI.
- **Improved Operational Efficiency:** BI systems can identify bottlenecks and inefficiencies in processes. For example, a BI report might show that a particular stage in the supply chain consistently causes delays, allowing managers to address it directly.
- **Gain Competitive Advantage:** By understanding market trends, customer behavior, and their own performance better than their competitors, companies can spot opportunities and threats early and act swiftly.

The Need for Business Intelligence

- **Identify New Revenue Opportunities:** Analyzing sales and customer data can reveal untapped markets, potential for upselling/cross-selling, or new product ideas that management hadn't considered.
- **Understand Customer Behavior:** BI helps in segmenting customers, analyzing purchasing patterns, and measuring customer satisfaction. This leads to more effective marketing, improved customer service, and higher retention rates.
- **Track Key Performance Indicators (KPIs):** BI allows organizations to monitor their health in real-time by tracking KPIs like sales growth, customer acquisition cost, employee productivity, and profit margins on centralized dashboards.
- **Regulatory Compliance and Risk Management:** For many industries, reporting to regulatory bodies is mandatory. BI systems can streamline this process. They also help in identifying potential risks, such as a sudden drop in sales in a key region or a rising debtors' list.

Key Terms Used in BI

- Database
 - Relational
 - NOSQL
- **Data Warehouse:** A central repository where data from various operational systems (like sales, marketing, HR) is stored, cleaned, and integrated. It is designed specifically for query and analysis, not for transaction processing.
- **Data Mart:** A subset of a data warehouse, designed for a specific line of business (e.g., a data mart for the sales department or for finance). It's smaller and more focused.

Key Terms Used in BI

- **OLAP (Online Analytical Processing):** A technology that allows users to analyze multidimensional data from multiple perspectives. For example, a user can view sales data by *region*, by *time*, by *product*, and by *salesperson* all at once.
- **Dashboard:** A graphical user interface that provides an at-a-glance view of key performance indicators (KPIs) and metrics, often using charts, graphs, and gauges.
- **KPI (Key Performance Indicator):** A measurable value that demonstrates how effectively a company is achieving key business objectives. Examples: Monthly Recurring Revenue, Customer Churn Rate, Net Promoter Score (NPS).
- **Data Mining:** The process of discovering patterns, correlations, and anomalies in large datasets using machine learning, statistics, and database systems.

Key Terms Used in BI

- **Ad-hoc Reporting:** The process of creating custom, on-the-fly reports to answer a specific business question, without having to use a pre-built, standard report.
- **Big Data:** Extremely large and complex datasets that are difficult to process using traditional data processing applications. BI tools are increasingly incorporating big data analytics.
- **ELT(Extract Load and Transform)**
- **ETL (Extract, Transform, Load):** The three-step process that is fundamental to data warehousing.
 - **Extract:** Data is pulled from different source systems (e.g., CRM, ERP, Excel sheets).
 - **Transform:** Data is cleaned, standardized, deduplicated, and reformatted into a consistent structure.
 - **Load:** The transformed data is loaded into the data warehouse.

General concept of Data Warehouse

- Data Warehouse
- History of Data Warehousing
- Need for Data Warehouse
- Data Warehouse Architecture
- Data Mining Works with DWH
- Features of Data warehouse
- Data Mart
- Application Areas

Online Analytical Processing (OLAP)

- Online Analytical Processing (OLAP)
- Nature of OLAP analysis
- Types of OLAP
- OLAP Tools
- OLTP and OLAP Capgemini Public
- OLAP Functional requirements
- OLAP Fast and Selective
- Operational versus Informational System

Online Analytical Processing (OLAP)

- OLAP is a category of software technology that enables analysts, managers, and executives to gain insight into data through fast, consistent, interactive access to a wide variety of possible views of information.
- This information has been transformed from raw data to reflect the real-dimensional understanding of the enterprise.

Nature of OLAP Analysis

- **Slice:** Selecting one specific dimension from the cube, creating a sub-cube.
 - *Example:* "Show me all data for Year = 2023." This is like taking a single, flat slice through the time dimension.
- **Dice:** Selecting a specific sub-set of data across multiple dimensions.
 - *Example:* "Show me data for Product Category = 'Electronics' and Location = 'Europe' and Time = Q1 & Q2." You are carving out a smaller cube from the larger one.

Nature of OLAP Analysis

- **Drill Down/Up:** Navigating through levels of a dimension hierarchy.
 - **Drill Down:** Going from a summary to more detailed data.
 - *Example:* Year Total -> Quarterly Sales -> Monthly Sales.
 - **Drill Up (or Roll-Up):** The opposite. Going from detailed data to a summarized level.
 - *Example:* Sales by City -> Sales by State -> Sales by Country.
- **Pivot (or Rotate):** Changing the dimensional orientation of a report or view.
 - *Example:* Swapping the rows and columns in a report. What was on the rows (Products) moves to the columns, and what was on the columns (Time) moves to the rows. This provides a different perspective on the same data.

Types of OLAP

- Multidimensional OLAP (MOLAP)
 - The classic approach. Data is stored in a proprietary, pre-calculated, multidimensional format (the "cube").
- Relational OLAP (ROLAP)
 - Works directly with the relational database (the data warehouse). It uses a star or snowflake schema and dynamically generates SQL queries to perform calculations.
- Hybrid OLAP (HOLAP):
 - A combination of MOLAP and ROLAP. It stores aggregate data in a MOLAP cube for speed and keeps detailed data in the relational database.

OLAP Tools

- **Microsoft Power BI**
- **Tableau**
- **Qlik Sense**
- **Microsoft Analysis Services (SSAS)**
- **Oracle Essbase**
- **SAP BW/4HANA**

OLTP vs. OLAP

15 mins

- **OLAP Functional Requirements (The "CORE" Requirements)**
- **OLAP: Fast and Selective**
- **Operational vs. Informational System**

Data Mining

- Data Mining is the process of discovering hidden, previously unknown, and potentially useful patterns, correlations, and knowledge from large volumes of data.
- It goes beyond mere data observation or querying; it involves using sophisticated algorithms and statistical techniques to **predict trends and behaviors** and to **identify relationships** that are not immediately obvious.

The Knowledge Discovery Process (KDD)

- Data Mining is not a single step; it is a crucial part of the larger Knowledge Discovery in Databases (KDD) process. KDD is the overall process of converting raw data into useful knowledge.
- **Data Cleaning**
- **Data Integration**
- **Data Selection**
- **Data Transformation**
- **Data Mining**
- **Pattern Evaluation**
- **Knowledge Presentation**

The Need for Data Mining

- **Information Overload:** The volume of data is too massive for manual analysis.
- **Automated Pattern Discovery:** Humans are good at spotting patterns in 2 or 3 dimensions, but data exists in dozens of dimensions. Algorithms can find complex, multi-dimensional patterns we would never see.
- **Competitive Advantage:** The ability to predict customer behavior, optimize operations, and mitigate risks before competitors do is a massive business advantage.
- **Decision Support:** Moving from intuition-based to data-driven decision-making.

- Use of Data mining
- Data mining and Business Intelligence
- Types of data used in Data mining

Types of Data Used in Data Mining

- **Relational Databases:** The most common source, with tables, rows, and columns.
- **Data Warehouses:** The ideal source, as data is already cleaned and integrated.
- **Transactional Data:** Records of transactions (e.g., sales, logs).
- **Flat Files:** CSV, Excel spreadsheets.
- **Multimedia Data:** Images, audio, video.
- **Text Data (Text Mining):** Documents, emails, web pages, social media posts.
- **Web Data (Web Mining):** Clickstream data from web servers.
- **Time-Series Data:** Data points indexed in time order (e.g., stock prices, sensor data).

Data Mining Applications (By Industry)

- **Retail:**

- Market Basket Analysis for store layout and promotions.
- Customer segmentation for loyalty programs.
- Demand forecasting for inventory management.

- **Banking & Finance:**

- Credit scoring and risk analysis.
- Fraud detection in credit card and insurance claims.
- Algorithmic trading.

- **Telecommunications:**

- Churn prediction to identify customers likely to switch providers.
- Campaign optimization.

Data Mining Applications (By Industry)

- **Insurance:**

- Claims analysis for fraud detection.
- Customer profitability analysis.

- **Healthcare:**

- Predicting disease outbreaks.
- Identifying effective treatments for specific patient groups.

- **Manufacturing:**

- Predictive maintenance for machinery.
- Optimizing supply chains.

Data Mining Products

- **Standalone Open-Source Tools / Libraries:**
 - **R & R Libraries:** A powerful statistical programming language with packages like dplyr, caret, and randomForest for data mining.
 - **Python & Scikit-Learn:** The dominant language for data science, with Scikit-Learn being a premier library for machine learning. Also includes libraries like Pandas, NumPy, and TensorFlow.
 - **Weka:** A popular, free, GUI-based Java software suite for machine learning.

Data Mining Products

- **Integrated within Major BI Platforms:**

- **Microsoft SQL Server Analysis Services (SSAS):** Includes powerful data mining algorithms.
- **SAP BW/BI:** Integrated analytics capabilities.
- **Oracle Data Mining:** Algorithms built into the Oracle Database.

- **Cloud-Based AI/ML Platforms:**

- **Microsoft Azure Machine Learning**
- **Google Cloud AI Platform**
- **Amazon SageMaker**

These are the modern trend, offering scalable, managed services for building, training, and deploying data mining models.

ETL: The Engine of Data Integration

- ETL stands for **Extract, Transform, Load**. It is a fundamental process in data warehousing and BI that involves:
- **Extracting** data from heterogeneous source systems.
- **Transforming** it into a clean, consistent, and useful format.
- **Loading** it into a target data warehouse or data mart.
- The primary goal of ETL is to consolidate disparate data into a single, trusted source of truth that is optimized for analysis and reporting.

The ETL Process in Detail

- **Stage 1: Extract**
- This is the phase of data acquisition from various source systems.
- **Sources:** Operational databases (e.g., Oracle, SQL Server, MySQL), CRM systems (e.g., Salesforce), ERP systems (e.g., SAP), flat files (CSV, Excel), web logs, and APIs.
- **Methods:**
 - **Full Extract:** The entire dataset is copied from the source each time. Simple but can be resource-intensive for large datasets.
 - **Incremental/Delta Extract:** Only the data that has changed since the last extraction is captured. This is more efficient and is common in production systems. It often uses timestamps, logs, or incrementing keys to identify changes.

Stage 2: Transform

- This is the most complex and critical stage where data is cleansed, standardized, and prepared for analysis. Common transformation tasks include:
- **Data Cleansing:** Correcting inconsistencies and errors (e.g., standardizing "NY," "New York," "N.Y." to a single value).
- **Data Validation:** Enforcing data integrity rules and rejecting/filtering invalid records.
- **Deduplication:** Identifying and removing duplicate records.
- **Format Standardization:** Converting data types and formats (e.g., converting MM/DD/YYYY to YYYY-MM-DD).
- **Calculations and Derivation:** Creating new calculated fields (e.g., Profit = Revenue - Cost).
- **Aggregation:** Summarizing data (e.g., daily sales totals to monthly totals).
- **Joining and Merging:** Combining data from multiple sources or tables.
- **Splitting Columns:** Breaking a single column into multiple ones (e.g., splitting "Full Name" into "First Name" and "Last Name").
- **Pivoting/Unpivoting:** Changing the structure of the data for better storage in the data warehouse.

Stage 3: Load

- This is the process of writing the transformed data into the target data warehouse.
- **Methods:**
 - **Full Load:** The entire target table is erased and reloaded with the new data. Used for small dimensions or during initial loads.
 - **Incremental Load:** Only new, changed, or updated records are inserted into the target table. This is the standard for large fact tables to maintain performance.
- **Frequency:** The load can be scheduled (e.g., nightly, weekly) or happen in near real-time (a process often called ELT or Change Data Capture).

Metadata Used in ETL

- **Source Metadata:**
 - Source system names, connection strings, and types (e.g., SQL Server, Oracle).
 - Table and column names from the source.
 - Data types and formats in the source.
- **Transformation Metadata (Business Rules):**
 - The logic for each transformation (e.g., "Trim whitespace from Customer_Name").
 - Data validation rules (e.g., "Email must contain an '@' symbol").
 - Mapping documents that define how source fields relate to target fields.
- **Process Execution Metadata:**
 - **Job Logs:** Start time, end time, and status (Success/Failure) of each ETL job.
 - **Performance Statistics:** Number of rows extracted/transformed/loaded, processing time for each step.
 - **Error Logs:** Details of any records that failed validation or processing, including the error reason. This is vital for data quality management.

Metadata in Data Warehousing

- **Technical Metadata:**

- **Purpose:** Used by IT staff and database designers to manage and maintain the warehouse.
- **Includes:**
 - Table structures, data types, and primary/foreign key relationships in the warehouse.
 - ETL process details (as described above).
 - Partitioning strategies, indexing schemes, and storage information.

Metadata in Data Warehousing

- **Business Metadata:**

- **Purpose:** Used by business users to understand and trust the data they are analyzing.
- **Includes:**
 - Business definitions of terms (e.g., "What is the exact business definition of an 'Active Customer'?").
 - Data owner information (who is responsible for the data).
 - Calculated field logic (e.g., "Profit is defined as Revenue minus Cost of Goods Sold and Marketing Expenses").
 - Report and dashboard descriptions.

Metadata in Data Warehousing

- **Operational Metadata:**

- **Purpose:** Provides information about the operation and usage of the data warehouse.
- **Includes:**
 - Data lineage: Shows the full flow of data from its source to its current form in a report. (e.g., "This KPI on the CEO's dashboard comes from this column in the fact table, which was loaded by this ETL job from this source system").
 - Data currency: When was the data last updated? Is this a live connection or a nightly snapshot?
 - Access patterns and performance statistics: Who is using what data and how often?

Simple Data Warehouse Model

- The most common and simplest data warehouse model is the **Dimensional Model**, which is designed for high-performance querying and ease of use for business users. It typically consists of two types of tables:
 - Fact Table
 - Dimension Table

Fact Tables

- These are the primary tables that store the **measurable, quantitative data** about business operations (the "verbs").
- **Structure:** They are often the largest tables.
 - **Foreign Keys:** Connections to dimension tables.
 - **Measures/Facts:** The numerical values to be analyzed (e.g., Sales Amount, Quantity Sold, Profit).
- **Example:** A Fact_Sales table would contain keys like ProductKey, DateKey, StoreKey and measures like SalesAmount.

Database

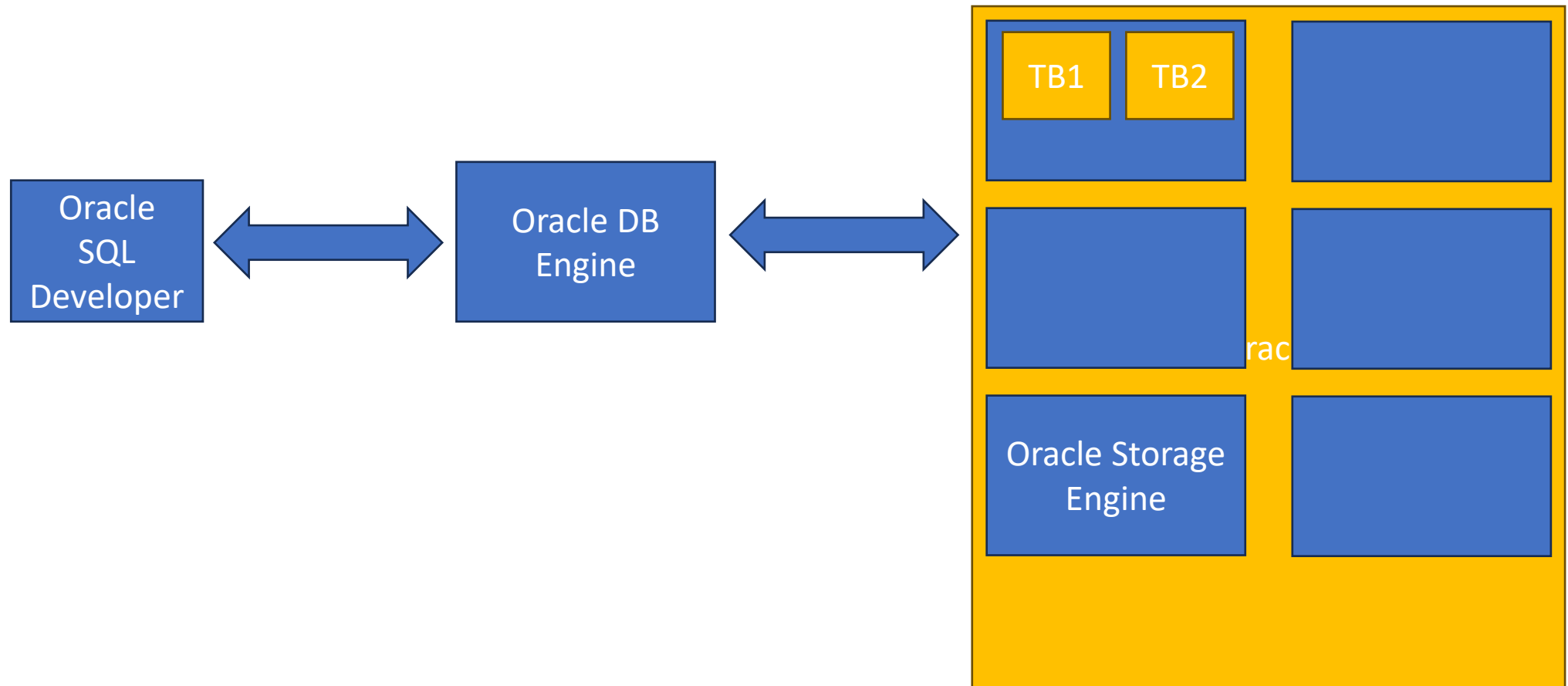
- A database system is typically divided into two main **architectural components**:
- **The Database Engine (DBMS Core)**
- **The Storage Engine (Storage Subsystem)**

- **InnoDB (Default)**
- **MyISAM(historical)**

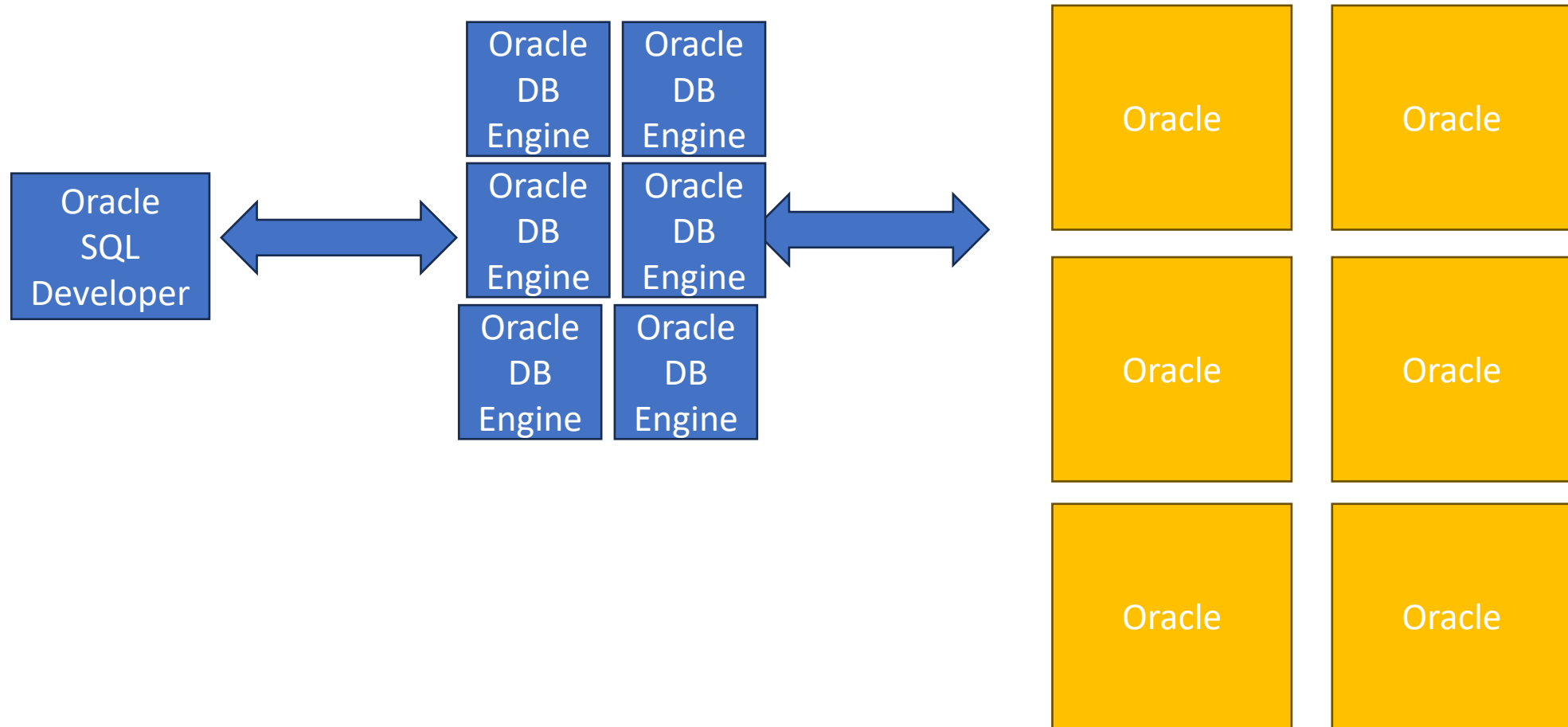
Database

- Client Layer
- DB Engine
- Storage Engine

Database

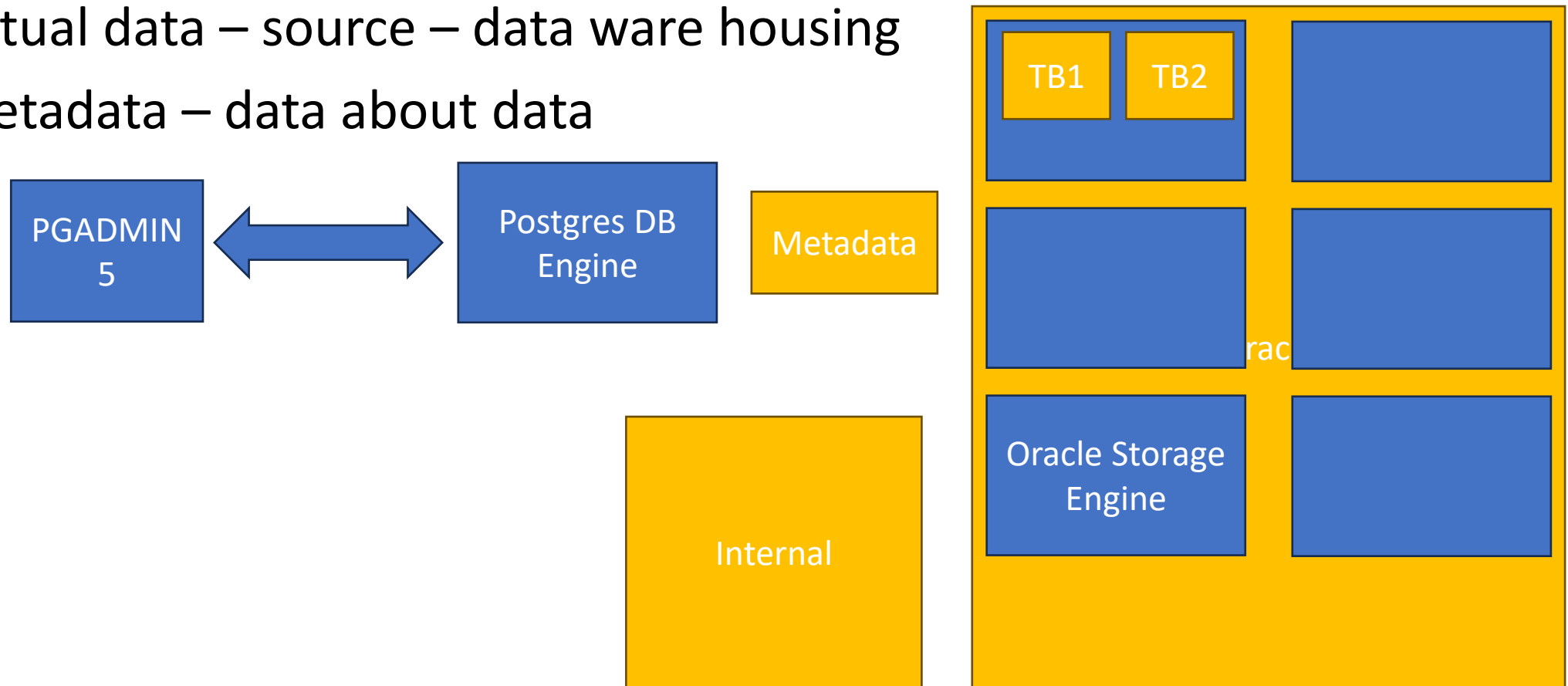


Database



Data Warehouse – External Storage

- Actual data – source – data ware housing
- Metadata – data about data



Data Warehousing

- Client Layer
- DB Engine
- Metadata Layer
- Storage Engine

- A **schema** is the **blueprint** or **structure** of a database.
- It defines how data is organized — what tables exist, what columns they have, and how tables are related.
- Star Schema - One central **fact table** connected to multiple **dimension tables**.
- Snowflake Schema - Like a star schema, but dimensions are further broken into sub-dimensions (more normalized).
- Galaxy Schema - Also called a "fact constellation" — multiple fact tables sharing dimension tables.
- A **schema** is the database's **skeleton** — the layout of tables and how they relate.



EHR



EMR



Clinical
systems



Extract
Transform
Load



Data
Warehouse



Business Intelligence

Data Mining



Business Analytics

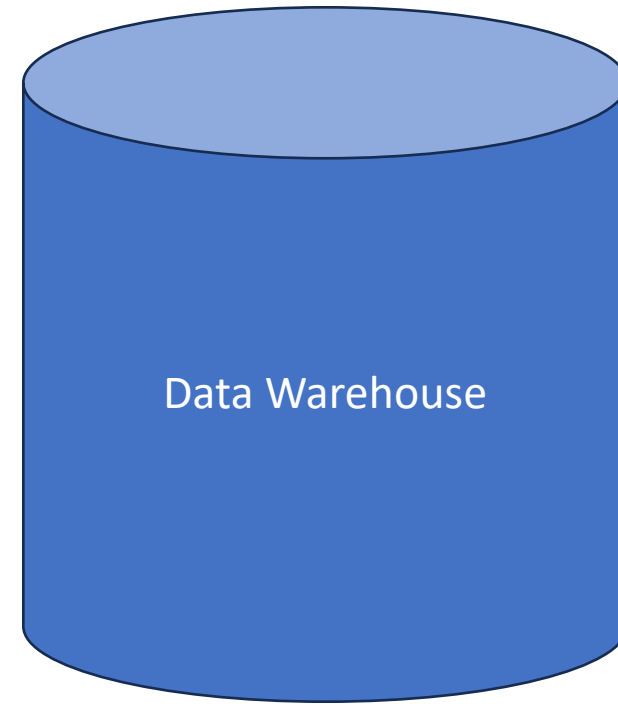
Big Data



Predictive Analytics

Reporting

- HR – 500GB Remaining – Employee System Logs



What is an ERD (Entity Relationship Diagram)?

- An **ERD** is a visual blueprint of a database. It's a flowchart that illustrates the logical structure of a database by showing:
 - The **entities** (like nouns, e.g., Student, Course).
 - The **attributes** of those entities (like adjectives, e.g., StudentID, CourseName).
 - The **relationships** between the entities (like verbs, e.g., "enrolls in").
- **Primary Purpose:** To provide a clear, graphical understanding of how data is organized and interconnected before a single line of code is written.
- It's a crucial tool for database designers, developers, and stakeholders to agree on the data requirements.

Why Do We Use ERDs? (The Use Cases)

- **Database Design & Blueprint:** It's the foundational step in designing a relational database, much like an architect's blueprint for a house.
- **Requirement Gathering:** In software engineering, ERDs help translate business needs (e.g., "students enroll in courses") into a structured data model.
- **Communication Tool:** It offers an easy-to-understand visual model for technical and non-technical team members to discuss data structures.
- **Database Implementation:** ERDs can be directly translated into SQL code to create tables, ensuring the physical database matches the intended design.
- **Understanding Data:** It helps designers see the "big picture" of the information the database will hold and how it all fits together.

The Core Building Blocks of an ERD

- **A. Entities**

- **Definition:** These are the main "objects" or "concepts" about which you want to store data. They are the "nouns" of the database.
- **Examples:** Student, Faculty, Course, Employee, Product.
- **In the Database:** Each entity becomes a **table**.

The Core Building Blocks of an ERD

- **Attributes**
- **Definition:** These are the properties or characteristics that describe an entity. They are the "adjectives."
- **Examples:** For a Student entity, attributes could be StudentID, FirstName, LastName, Email.
- **In the Database:** Attributes become the **columns** of a table.
- **Special Types of Attributes:**
 - **Primary Key (PK):** A unique identifier for *each record* in a table. It **MUST** be:
 - **Unique:** No two records can have the same value.
 - **Not Null:** It cannot be empty.
 - **Immutable:** It should never change.
 - **Example:** StudentID. This is why websites don't let you change your username if it's used as a PK.
 - **Foreign Key (FK):** An attribute in one table that refers to the Primary Key in another table. It creates the "link" between tables.
 - **It does NOT have to be unique.** Many students can have the same FacultyID (meaning they are in the same faculty).
 - **Example:** The FacultyID in the Student table is a Foreign Key referencing the Faculty table.

The Core Building Blocks of an ERD

- **Relationships**
- **Definition:** These describe how entities are connected to each other. They are the "verbs."
- **Examples:** A Student *enrolls in* a Course. A Faculty *provides* a Course.

Cardinality: The "Rules" of Relationships

- Cardinality defines the numerical relationship between instances of one entity and instances of another. It's the most critical part of defining business rules in an ERD.
- **One-to-Many (1:M) / Many-to-One (M:1)**
- **Description:** One record in Entity A can be associated with *many* records in Entity B, but a record in Entity B is associated with only *one* record in Entity A.
- **How to Implement:** The Primary Key from the "one" side (Table A) is placed as a Foreign Key on the "many" side (Table B).
- **Examples from your presentation:**
 - **One Faculty has Many Students:** (1 Faculty : M Students). A student cannot be in both Engineering and Law simultaneously. FacultyID (PK from Faculty) is placed in the Student table as an FK.
 - **One Faculty offers Many Courses:** (1 Faculty : M Courses). A specific "Chemistry" course belongs to only one faculty.
 - **Real-world example:** One Customer can place Many Orders, but a specific Order is placed by only One Customer.

Cardinality: The "Rules" of Relationships

- **Many-to-Many (M:N)**
- **Description:** One record in Entity A can be associated with *many* records in Entity B, and vice-versa.
- **The Challenge:** This cannot be directly implemented in a relational database with just two tables.
- **The Solution: A Bridge Table (or Junction Table).**
 - You create a new intermediate table (e.g., Enrollment).
 - This bridge table has Foreign Keys that reference the Primary Keys of the two original tables.
 - **Example:** The Enrollment table has StudentID (FK) and CourseID (FK). This allows you to record that Student A is in Courses 101, 102, and 103, and that Course 101 has Students A, B, and C.
 - **Primary Key for Bridge Table:** Often, the combination of the two Foreign Keys (StudentID, CourseID) forms a **Composite Primary Key**, as this combination is unique.

Cardinality: The "Rules" of Relationships

- **One-to-One (1:1)**
- **Description:** One record in Entity A is associated with exactly one record in Entity B, and vice-versa.
- **Less common**, often used for security or to split a large table.
- **Example:** One Person has exactly one Passport, and one Passport belongs to exactly one Person.

- **Bill Inmon's Top-Down Approach**
- **Ralph Kimball's Bottom-Up Approach**

Dimensional Modelling

- **The Core Problem:** Business users cannot and should not need to write complex SQL with 10+ table joins from a normalized database.
- Dimensional Modeling. It is a **data structure technique** specifically designed for data warehousing and BI tools. Its primary goal is to make data **easy to understand and query** for end-users, while delivering **high performance** for read operations.
- **The Fundamental Building Blocks: Facts and Dimensions**

Why is Dimensional Modeling So Important? (The Advantages)

- **Ease of Use:** The model is intuitive. Business users can easily understand "Sales by Product Category" or "Profit by Region and Year" because it maps directly to how they think.
- **Query Performance:**
 - **Fewer Joins:** A star schema requires far fewer joins than a normalized 3NF model.
 - **Optimized for SELECTs:** Denormalization and predictable query patterns allow databases to create highly efficient execution plans.
- **Adaptability to Change:** You can add new dimensions or new attributes to existing dimensions without breaking existing queries on the fact table.
- **Standard for BI Tools:** Nearly all BI tools (Tableau, Power BI, Looker) are designed to work seamlessly with star schemas. They can automatically detect relationships and generate efficient queries.
- **Historical Tracking:** Dimensions allow you to track history (e.g., what was a customer's segment *at the time of the sale?*), which is crucial for accurate historical analysis.

Star Schema: The Gold Standard for Performance

- **Structure:** One central fact table connected directly to multiple dimension tables.
- **Key Characteristics:**
 - **Fully Denormalized:** Each dimension table contains all attributes, even if redundant
 - **Simple Joins:** Only one join between fact table and any dimension table
 - **Optimized for Query Performance:** Minimal joins = faster queries
 - **Business-Friendly:** Easy for users to understand and navigate

Star Schema: The Gold Standard for Performance

- **Advantages:**
 -  **Best query performance** (fewer joins)
 -  **Simplest for business users** to understand
 -  **Easy to implement** and maintain
 -  **BI tools work excellently** with this structure
- **Disadvantages:**
 -  **More storage space** due to data redundancy
 -  **Data integrity** relies on ETL processes
 -  **Updates** can be more complex (need to update multiple rows)

Snowflake Schema: The Normalized Approach

- **Structure:** A star schema where dimension tables are normalized into multiple related tables.
- **Key Characteristics:**
 - **Normalized Dimensions:** Dimension tables are broken down to eliminate redundancy
 - **Multiple Joins Required:** Need to join through hierarchy to get all attributes
 - **Reduced Storage:** Less data redundancy
 - **Better Data Integrity:** Follows database normalization principles

Snowflake Schema: The Normalized Approach

- **Advantages:**
-  **Less storage space** (reduced redundancy)
-  **Better data integrity** (normalized structure)
-  **Easier updates** (change data in one place only)
-  **Easier to add** new hierarchical levels
- **Disadvantages:**
-  **Poorer query performance** (more complex joins)
-  **More complex for business users** to understand
-  **Harder to maintain** (more tables to manage)
-  **BI tools may struggle** with complex relationships

Galaxy Schema (Fact Constellation): Multiple Fact Tables

- **Structure:** Multiple fact tables that share common dimension tables.
- **Key Characteristics:**
 - **Shared Dimensions:** Common dimensions used across multiple business processes
 - **Multiple Fact Tables:** Represents different business processes in one model
 - **Comprehensive View:** Provides enterprise-wide data integration
 - **Complex Structure:** Most complex of the three schemas

Galaxy Schema

- **Advantages:**
-  **Enterprise-wide view** of data
-  **Shared dimensions** ensure consistency
-  **Comprehensive analytics** across business processes
-  **Efficient storage** through dimension sharing
- **Disadvantages:**
-  **Most complex** to design and implement
-  **Difficult to maintain**
-  **Requires careful planning** for shared dimensions
-  **Can become unwieldy** if not properly managed

Star Schema

- The star schema is a denormalized structure with a central fact table surrounded by multiple dimension tables. It's favored in BI because it's simple, fast to query, and easy to understand. Here's how your HR scenario would look:
- Star Schema Diagram (Conceptual)
- Fact Table:
 - Stores business events/metrics: (e.g., salary, bonus, performance_rating, hire/termination dates)
- Dimension Tables:
 - Employee Dimension (employee_id, name, ssn, gender, birth_date, contact info, address)
 - Department Dimension (department_id, department_name, manager info)
 - Job Dimension (job_id, job_title, commission_pct)
 - Date Dimension (date_id, actual date, year, month, quarter) for hire/termination
 - Project Dimension (project_id, project_name) — skills and projects should be pulled into separate tables to satisfy 1NF

Slowly Changing Dimensions (SCDs)

- **SCDs** are techniques to handle changes in dimension attribute values over time while preserving historical accuracy.
- They answer the critical question: *"How do we track changes in descriptive data so that historical reports remain accurate?"*
- **SCD Type 1: Overwrite History**
- **SCD Type 2: Preserve Full History**
- **SCD Type 3: Limited History**

Type 1: Update Changes

Supplier_Key	Supplier_Code	Supplier_Name	Supplier_State
123	ABC	Acme Supply Co	CA



Supplier_Key	Supplier_Code	Supplier_Name	Supplier_State
123	ABC	Acme Supply Co	IL

Type 2: Keep Historical

Supplier_Key	Supplier_Code	Supplier_Name	Supplier_State	Start_Date	End_Date
123	ABC	Acme Supply Co	CA	01-Jan-2000	21-Dec-2004
124	ABC	Acme Supply Co	IL	22-Dec-2004	

Type 3: Preserve Limited History

Supplier_Key	Supplier_Code	Supplier_Name	Original_Supplier_State	Effective_Date	Current_Supplier_State
123	ABC	Acme Supply Co	CA	22-Dec-2004	IL